

Informe de Laboratorio 10

Tema: Frontend

Nota

Estudiantes	Escuela	Asignatura
Gustavo Alonso Carrillo Villalta Fredy José Aragón Carpio Diego Marcelo Arce Coaquira José Manuel Bravo Rojas	Escuela Profesional de Ingeniería de Sistemas	Desarrollo de Aplicaciones Web Semestre: I Código: 2025004 Grupo: Desarrolladores

Laboratorio	Tema	Duración
10	Frontend	10 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - A	15 de Junio de 2026	22 de Junio de 2026

1. Laboratorio 10: Django REST Framework y Frontend (Consumo de API)

1.1. Descripción y Objetivos

En esta práctica, el enfoque principal ha sido la integración del backend (desarrollado en sesiones previas con DRF) con una interfaz frontend, asegurando una comunicación eficiente mediante JSON anidados y políticas de seguridad robustas. Los objetivos fueron:

- Implementar **JSON anidados estratégicos** para optimizar la transferencia de datos y reducir el número de peticiones HTTP.
- Configurar **permisos de acceso** para permitir operaciones de lectura (GET) de forma anónima, mientras se protegen las operaciones de modificación (POST, PUT, DELETE) mediante autenticación robusta.
- Registrar y estructurar los **endpoints** necesarios para el consumo de la API desde el frontend.
- Realizar el **despliegue del sistema** completo (backend y frontend) en plataformas de nube (CDN/Hosting), garantizando su disponibilidad pública.
- Documentar el proceso de integración y el flujo de comunicación entre capas mediante un archivo README.md y capturas de pantalla de la API y la interfaz de usuario.

1.2. Resumen del Ejercicio

Durante el desarrollo del laboratorio, se configuró el proyecto para soportar serializadores anidados, lo que permitió exponer objetos relacionales complejos en estructuras JSON unificadas. Se ajustó la configuración de permisos en `views.py` y `settings.py` para diferenciar el nivel de acceso a los datos, delegando la responsabilidad de la integridad de la información al backend.

Posteriormente, se desarrolló el frontend consumiendo estos servicios REST y se procedió con la puesta en producción en plataformas cloud, cumpliendo con los estándares de despliegue continuo requeridos. Este ejercicio permitió consolidar el flujo completo de una arquitectura web moderna, desde la modelación de datos hasta la interfaz de usuario final.

1.3. Entregables

El proyecto se documenta y entrega a través de:

- **Repositorio en GitHub:** <https://github.com/FredyAragon/Cafe-Sys>
- **URL Video Demostración:** https://youtu.be/ej-uRl_kEQ4
- **URL Backend Desplegado:** <https://cafesys-backend.onrender.com>
- **URL Frontend Desplegado:** <https://cafe-sys.vercel.app/>
- **Informe PDF:** <https://github.com/FredyAragon/Cafe-Sys/tree/main/informes/Informe-Lab10-DAW.pdf>

1.4. 1.4. Instalación y configuración del framework Angular

Para el desarrollo del frontend de *Cafe-Sys*, se utilizó Angular en su versión 21. La instalación y configuración se realizó siguiendo estándares de desarrollo profesional, utilizando la CLI (Command Line Interface) de Angular para garantizar la estructura modular del proyecto.

- **Requisitos previos:** Se verificó la instalación de Node.js (entorno de ejecución para JavaScript) y NPM (gestor de paquetes) para gestionar las dependencias del proyecto.
- **Instalación de Angular CLI:** Se utilizó el comando global para acceder a las herramientas de construcción:

```
npm install -g @angular/cli@21.1.0
```

- **Inicialización del proyecto:** Se ejecutó el comando de creación, seleccionando la configuración de *Standalone Components* para una arquitectura moderna y eficiente:

```
ng new frontend --standalone --routing --style=css
```

- **Configuración de herramientas de desarrollo:** Se integraron las herramientas de testing *vitest* y *jsdom*, configurando el archivo `vite.config.ts` para asegurar que el sistema de compilación fuera compatible con la nueva arquitectura de Angular.
- **Instalación de dependencias:** Finalmente, se procedió con la instalación de los paquetes definidos en el `package.json`, asegurando la compatibilidad entre el *Core* de Angular y las librerías de terceros (RxJS, etc.).

Esta configuración garantiza que el entorno de desarrollo sea escalable, rápido y fácil de desplegar en entornos de integración continua (CI/CD).

2. Equipos, materiales y temas utilizados

Esta sección detalla las librerías y dependencias esenciales para el funcionamiento de *Cafe-Sys*, divididas por su entorno de ejecución.

2.1. Definición y Explicación de las Librerías Utilizadas

2.1.1. Backend (Python/Django)

- **Django (6.0.5)**: Framework web de alto nivel que fomenta el desarrollo rápido. Proporciona la estructura MVC, ORM para interacción con PostgreSQL y sistemas de autenticación integrados.
- **Django REST Framework (3.17.1)**: Biblioteca flexible para construir APIs RESTful. Es esencial para la creación de serializadores que transforman modelos a JSON y la gestión de permisos en los endpoints.
- **Simple JWT (5.3.1)**: Plugin de autenticación que implementa JSON Web Tokens, permitiendo una gestión de sesiones *stateless* ideal para aplicaciones SPA como la desarrollada en Angular.
- **django-cors-headers (4.9.0)**: Añade cabeceras CORS a las respuestas HTTP, permitiendo que el frontend (en un dominio distinto) consuma la API sin bloqueos de seguridad.
- **Gunicorn (23.0.0) y Whitenoise (6.9.0)**: Gunicorn actúa como el servidor WSGI en producción para manejar peticiones concurrentes, mientras que Whitenoise permite servir archivos estáticos directamente desde la aplicación.

2.1.2. Frontend (Angular/TypeScript)

- **Angular Core/Common/Compiler (21.1.0)**: Núcleo del framework que gestiona la inyección de dependencias, detección de cambios y compilación de plantillas optimizadas para el navegador.
- **Angular Forms (21.1.0) y Router (21.1.0)**: Módulos críticos para el manejo de formularios (reactivos y dirigidos por plantilla) y la navegación entre vistas dentro de la Single Page Application (SPA).
- **RxJS (7.8.0)**: Biblioteca de programación reactiva para manejar flujos de datos asíncronos, fundamental para gestionar las peticiones HTTP hacia el backend de Django.
- **TypeScript (5.9.2)**: Lenguaje de tipado estático que mejora la mantenibilidad y detección de errores en el desarrollo del frontend.

2.2. Resumen de Tecnologías

Categoría	Backend	Frontend
Framework	Django, DRF	Angular 21
Autenticación	Simple JWT	Manual (LocalStorage)
Base de Datos	PostgreSQL	N/A
Testing	N/A	Vitest, JSDOM

Cuadro 1: Resumen del stack tecnológico de Cafe-Sys.

2.3. Infraestructura

- **Docker**: Utilizado para la contenerización mediante `Dockerfile` y `docker-compose.yml`, garantizando la portabilidad del entorno de desarrollo y producción.
- **PostgreSQL**: Motor de base de datos relacional robusto donde reside toda la persistencia del sistema, conectado vía `psycopg2-binary`.

3. URL del Repositorio en GitHub

El laboratorio desarrollado se encuentra disponible en el siguiente repositorio de GitHub:

- **Repositorio:** <https://github.com/FredyAragon/Cafe-Sys>

4. Desarrollo de actividades del laboratorio

4.1. Backend: Permisos y Vistas

Para cumplir con el requisito de seguridad, se implementó una política de acceso diferenciada. La vista `OrdersViewSet` permite el consumo público mediante el método `GET` (lectura), pero restringe las operaciones de creación o modificación (`POST/PUT/DELETE`) exclusivamente a usuarios autenticados mediante `JWT`.

Listing 1: Vistas con permisos diferenciados

```
class OrdersViewSet(viewsets.ModelViewSet):
    permission_classes = [IsAuthenticated]
    queryset = Orders.objects.select_related('user', 'location').prefetch_related('details',
                                         'details__product').all()
    serializer_class = OrdersSerializer

    def get_permissions(self):
        # GET (list/retrieve) es publico, POST/PUT/PATCH requiere autenticacion
        if self.action in ('list', 'retrieve'):
            return [AllowAny()]
        return [IsAuthenticated()]
```

4.2. JSON Anidados Estratégicos

Para optimizar las consultas y reducir el tráfico de red, se utilizaron serializadores anidados. Esto permite que el frontend reciba en una sola petición el objeto principal y sus relaciones (como detalles de producto y usuario), evitando múltiples llamadas al servidor.

Listing 2: Serializadores anidados en serializers.py

```
class OrderDetailsNestedSerializer(serializers.ModelSerializer):
    product_detail = ProductDetailNestedSerializer(source='product', read_only=True)
    class Meta:
        model = OrderDetails
        fields = ('id', 'product', 'product_detail', 'quantity', 'subtotal')

class OrdersSerializer(serializers.ModelSerializer):
    details = OrderDetailsNestedSerializer(many=True, read_only=True)
    user_detail = UsersSerializer(source='user', read_only=True)
    # ... logica de creacion atmica ...
```

4.3. Frontend: Configuración y Consumo de API

El frontend se estructuró con Angular 21, utilizando dependencias optimizadas para una arquitectura SPA.

Listing 3: Dependencias principales en package.json

```
"dependencies": {
  "@angular/core": "^21.1.0",
  "@angular/forms": "^21.1.0",
  "@angular/router": "^21.1.0",
  "rxjs": "^7.8.0"
},
"devDependencies": {
  "vitest": "^4.0.8",
  "typescript": "^5.9.2"
}
```

La comunicación con el backend se centraliza en un servicio que gestiona las peticiones HTTP de forma asíncrona mediante **Observables**.

Listing 4: Consumo de API mediante HttpClient

```
@Injectable({ providedIn: 'root' })
export class ApiService {
  getOrders(): Observable<Order[]> {
    return this.http.get<Order[]>(`${this.API_URL}/orders/`)
      .pipe(timeout(this.TIMEOUT));
  }
}
```

4.4. Registro de URLs

Finalmente, se configuró el enrutamiento mediante **DefaultRouter**, lo que permite que DRF genere automáticamente las URLs estándar para todos los ViewSets registrados.

Listing 5: Registro de endpoints en urls.py

```
router = DefaultRouter()
router.register(r'users', UsersViewSet)
router.register(r'products', ProductsViewSet)
router.register(r'orders', OrdersViewSet)
# ... resto de rutas ...

urlpatterns = [
    path('token/', CustomTokenObtainPairView.as_view()),
    path('', include(router.urls)),
]
```

4.5. Implementación de Consulta Read-Only (GET)

Para validar que la vista de **Orders** permite el consumo público sin requerir token, se muestra la implementación del componente de consulta en el frontend, el cual interactúa directamente con el endpoint configurado como **AllowAny**. Desde la página de administrador se puede hacer el GET libremente.

Listing 6: Componente de lectura (GET) en el frontend

```
export class OrdenesComponent implements OnInit {
  private apiService = inject(ApiService);
  orders = signal<Order[]>([]);
}
```

```
loadOrders() {  
  // Esta petición no requiere de un token de autenticación  
  this.apiService.getOrders().subscribe({  
    next: (data: Order[]) => {  
      this.orders.set(data);  
    },  
    error: (err: any) => {  
      console.error('Error al cargar pedidos:', err);  
    }  
  });  
}
```

4.6. Implementación de Registro (POST) con Detalles

Por otro lado, para operaciones críticas como la creación de una orden, el sistema valida la sesión del usuario antes de ejecutar la petición POST. Este proceso encapsula la orden y sus detalles en una sola llamada atómica.

Listing 7: Método de registro (POST) con manejo de errores

```
finalizarCompra() {  
  // 1. Validación de sesión de usuario  
  const user = this.usuario();  
  if (!user) { this.error.set('Debes iniciar sesión.');
```



```
  // 2. Mapeo de productos a la estructura de la API  
  const detailsData = cartItems.map((item) => ({  
    product: item.product.id,  
    quantity: item.quantity,  
    unitPrice: Number(item.product.price)  
  }));  
  
  // 3. Petición POST atómica  
  this.apiService.createOrder({  
    user: user.id,  
    orderStatus: 'pending',  
    details_data: detailsData  
  }).subscribe({  
    next: (order: Order) => {  
      this.compraExitosa.set(true);  
      this.cartService.clear();  
    },  
    error: (err: any) => {  
      this.error.set('Error en la transacción');    }  
  });  
}
```

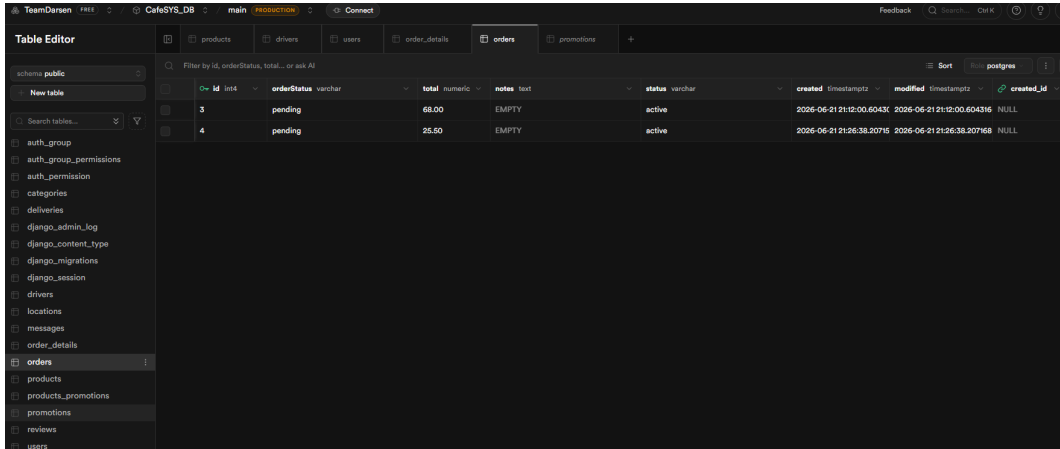
5. Resultados

La validación del sistema *Cafe-Sys* se realizó verificando la persistencia de los datos en la base de datos PostgreSQL alojada en Supabase, confirmando que las operaciones POST se registran correctamente

y que los datos recuperados mediante GET son consistentes.

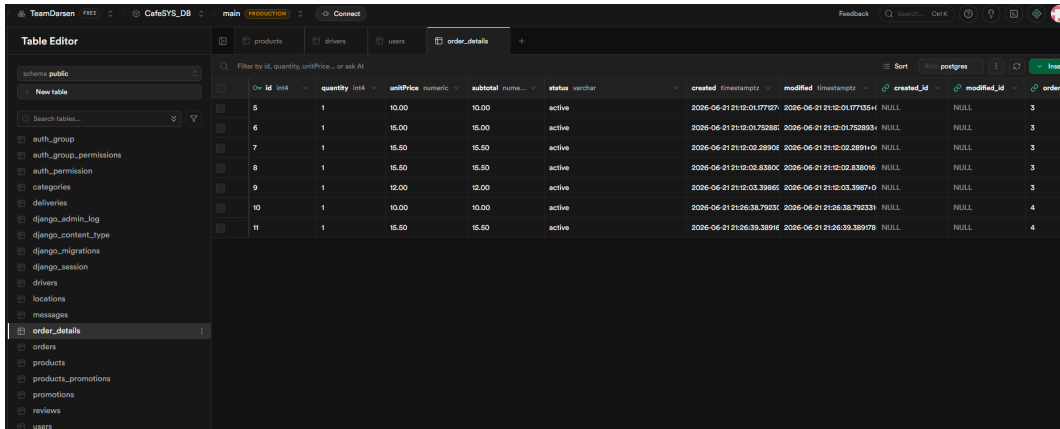
5.1. Validación de Persistencia (Supabase)

Las siguientes capturas muestran el estado de las tablas en Supabase tras ejecutar las operaciones CRUD desde el frontend.



id	orderStatus	total	notes	status	created	modified	created_id
3	pending	68.00	EMPTY	active	2026-06-21 21:12:00.60431	2026-06-21 21:12:00.604316	NULL
4	pending	25.50	EMPTY	active	2026-06-21 21:26:38.20745	2026-06-21 21:26:38.207468	NULL

Figura 1: Captura de la tabla *orders* en Supabase, reflejando el registro exitoso tras una petición POST.



id	quantity	unitPrice	subtotal	status	created	modified	created_id	modified_id	order_id
5	1	10.00	10.00	active	2026-06-21 21:12:01.77707	2026-06-21 21:12:01.777354	NULL	NULL	3
6	1	15.00	15.00	active	2026-06-21 21:12:01.75288	2026-06-21 21:12:01.7528934	NULL	NULL	3
7	1	15.50	15.50	active	2026-06-21 21:12:02.28908	2026-06-21 21:12:02.289140	NULL	NULL	3
8	1	15.50	15.50	active	2026-06-21 21:12:02.83800	2026-06-21 21:12:02.838016	NULL	NULL	3
9	1	12.00	12.00	active	2026-06-21 21:12:03.39865	2026-06-21 21:12:03.398710	NULL	NULL	3
10	1	10.00	10.00	active	2026-06-21 21:26:38.79231	2026-06-21 21:26:38.792333	NULL	NULL	4
11	1	15.50	15.50	active	2026-06-21 21:26:39.38916	2026-06-21 21:26:39.389178	NULL	NULL	4

Figura 2: Vista de la tabla *order_details* mostrando la integridad referencial y los datos anidados almacenados correctamente.

5.2. Validación de Consumo Frontend

Para demostrar que la API permite la lectura pública mientras protege la escritura, se presenta la interfaz del sistema:

CafeSys Admin

Employee

Dashboard

Órdenes

Administrador

ÓRDENES

Área de solo lectura para conductores y empleados.

Orden #4

Pending

S/. 25.50

6/21/20, 4:03PM

Cliente: Luis Arce (luis@gmail.com)

PRODUCTO	CANTIDAD	PRECIO UNITARIO	SUBTOTAL
Café Espresso	1	S/. 10.00	S/. 10.00
Café Intenso Pro 3	1	S/. 15.50	S/. 15.50

Orden #3

Pending

S/. 68.00

6/21/20, 4:12PM

Cliente: Luis Arce (luis@gmail.com)

PRODUCTO	CANTIDAD	PRECIO UNITARIO	SUBTOTAL
Café Espresso	1	S/. 10.00	S/. 10.00
Café Americano	1	S/. 15.00	S/. 15.00
Café Intenso Pro 3	1	S/. 15.50	S/. 15.50
Café Intenso Pro 3	1	S/. 15.50	S/. 15.50
Café Latte	1	S/. 12.00	S/. 12.00

Salir

Figura 3: Interfaz del sistema donde se listan las órdenes recuperadas, validando el funcionamiento del método GET (Read-Only).

5.3. Validación de Flujo en el Frontend

Para demostrar que el sistema no solo permite la consulta pública de datos, sino que también gestiona correctamente las operaciones de escritura (POST) bajo autenticación, se documenta el flujo de finalización de compra.

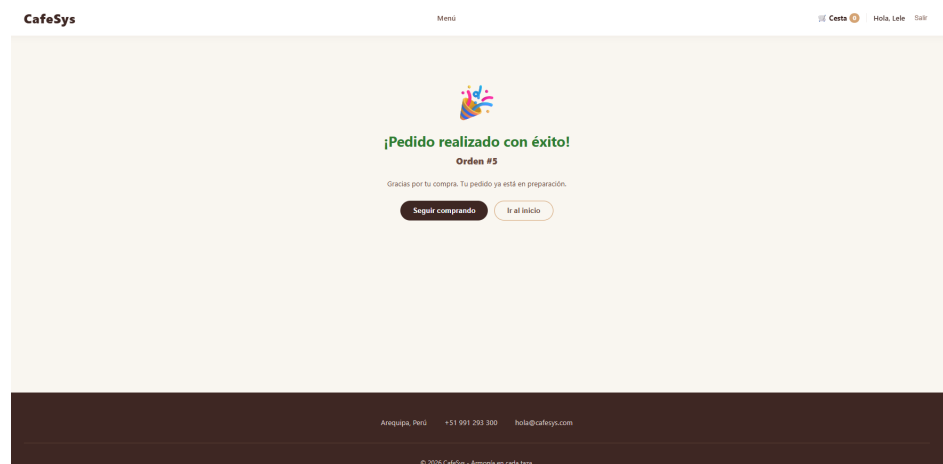
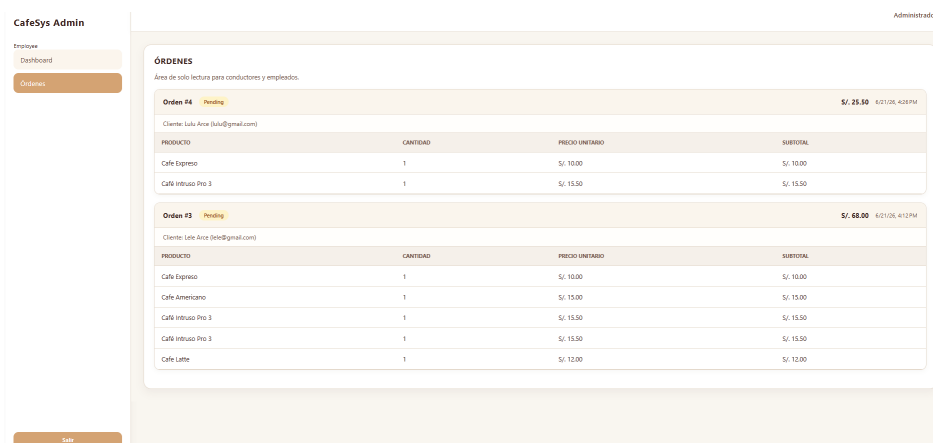


Figura 4: Interfaz del carrito de compras. Esta captura muestra la interacción del usuario justo antes de confirmar la compra (operación POST), la cual dispara la creación atómica de la orden y sus detalles en el backend.



PRODUCTO	CANTIDAD	PRECIO UNITARIO	SUBTOTAL
Café Espresso	1	S/ 10.00	S/ 10.00
Café Intenso Pro 3	1	S/ 15.50	S/ 15.50
Orden #4	Pending	S/ 25.50	6/21/20, 4:03 PM
Cliente: Luis Arce (luis@gmail.com)			
PRODUCTO	CANTIDAD	PRECIO UNITARIO	SUBTOTAL
Café Espresso	1	S/ 10.00	S/ 10.00
Café Americano	1	S/ 10.00	S/ 10.00
Café Intenso Pro 3	1	S/ 15.50	S/ 15.50
Café Intenso Pro 3	1	S/ 15.50	S/ 15.50
Café Latte	1	S/ 12.00	S/ 12.00
Orden #3	Pending	S/ 68.00	6/21/20, 4:12 PM
Cliente: Luis Arce (luis@gmail.com)			

Figura 5: Interfaz de listado de órdenes. Esta vista valida el correcto consumo del método GET (Read-Only), permitiendo visualizar los datos persistidos tras el proceso de compra.

6. Autoevaluación

La distribución porcentual refleja la participación activa y colaborativa de cada integrante en el desarrollo del sistema *Cafe-Sys*, abarcando desde la configuración de permisos y serializadores en el backend, hasta el consumo de la API y el despliegue del frontend.

- **Fredy José Aragón Carpio (100 %)**

Responsable de la configuración de los *endpoints* en Django REST Framework, la implementación de serializadores anidados para consultas complejas y la documentación técnica detallada en el informe.

- **Diego Marcelo Arce Coaquira (100 %)**

Encargado de la integración del *frontend* con Angular 21, el desarrollo de los servicios para el consumo de la API REST y creación de Frontend en general.

- **Gustavo Alonso Carrillo Villalta (100 %)**

Responsable de la definición de los esquemas de seguridad (*permission_classes*), pruebas de validación de datos en PostgreSQL y configuración del entorno de desarrollo.

- **José Manuel Bravo Rojas (100 %)**

Encargado de la gestión del despliegue en plataforma en la nube (CDN/Hosting).

El trabajo conjunto permitió integrar exitosamente la capa de seguridad (JWT + ReadOnly) con el flujo de datos del usuario, cumpliendo con todos los hitos establecidos en la rúbrica del laboratorio.

7. Rúbrica de Calificación

A continuación se presenta la rúbrica de evaluación para el Laboratorio 10, ajustada a los criterios de implementación y entrega.

Ítem	Descripción	Puntaje
package.json	Definición y explicación de las librerías utilizadas.	4
Configuración	Instalación y configuración detallada del framework frontend.	4
Componentes	Creación de componentes (vistas) para la interfaz de usuario.	4
Informe	Informe detallado de todos los pasos para el desarrollo de la práctica.	4
Deploy	URLs del backend y frontend disponibles en plataforma en la nube.	3
Prueba	Cumplimiento de recomendaciones y trabajo en equipo.	-0
Total		19

Cuadro 2: Rúbrica de evaluación oficial para el Laboratorio 10.